

Question 1 推导递归最小二乘法

最小二乘法，是一个用已知数据推导线性模型参数的算法。但是由于其本身运算量过大，对数据实时更新，需要同步实时更新参数的场景中，最小二乘法在迭代过程中会计算复杂度过高，影响算法效率。于是推导出了递归最小二乘法，根据新的数据点快速调整参数，使得过去所有数据点的预测误差平方和最小。

最小二乘法：

$$\begin{pmatrix} y_1 \\ y_2 \\ \dots \\ y_K \end{pmatrix} = \begin{pmatrix} x_1^1 & x_1^2 & \dots & x_1^M \\ x_2^1 & x_2^2 & \dots & x_2^M \\ \dots & \dots & \dots & \dots \\ x_K^1 & x_K^2 & \dots & x_K^M \end{pmatrix} \begin{pmatrix} \omega_1 \\ \omega_2 \\ \dots \\ \omega_M \end{pmatrix}$$

$$\mathbf{y}_K = \Phi_K \cdot \omega$$

其中 K 是数据的数量； M 是数据的特征维度； $\mathbf{y}_K$ 是数据的输出量； Φ_K 是数据的输入量，每行为一组数据在不同特征维度上的值； ω_K 是权重矢量； $y_K = \sum_{j=1}^M x_K^j \cdot \omega_j$

求解最小二乘法等价于

$$\begin{aligned} \mathbf{y}_K - \Phi_K \cdot \omega &= 0 \\ \Phi_K^T \cdot (\mathbf{y}_K - \Phi_K \cdot \omega) &= 0 \\ \Phi_K^T \cdot \mathbf{y}_K - \Phi_K^T \cdot \Phi_K \cdot \omega &= 0 \\ \omega &= (\Phi_K^T \cdot \Phi_K)^{-1} \cdot (\Phi_K^T \cdot \mathbf{y}_K) \end{aligned}$$

以得到线性模型的参数

递归最小二乘法的目标便是能够通过新的数据来更新 ω ，即写作如下的形式

$$\begin{aligned} \omega_K &= \omega_{K-1} + \epsilon \\ \omega_K &= (\Phi_K^T \cdot \Phi_K)^{-1} \cdot (\Phi_K^T \cdot \mathbf{y}_K) \end{aligned}$$

ω_{K-1} 是通过 $K-1$ 组数据得到的解，是已知值，我们的目标是通过新数据 $(\mathbf{x}_K, y_K)$ 来求解 ϵ

$$\begin{aligned} \Phi_K^T \cdot \Phi_K &= \begin{pmatrix} \mathbf{x}_1^T & \mathbf{x}_2^T & \dots & \mathbf{x}_K^T \end{pmatrix} \cdot \begin{pmatrix} \mathbf{x}_1 \\ \mathbf{x}_2 \\ \dots \\ \mathbf{x}_K \end{pmatrix} \\ &= \sum_{i=1}^{K-1} \mathbf{x}_i^T \cdot \mathbf{x}_i + \mathbf{x}_K^T \cdot \mathbf{x}_K \end{aligned}$$

$$\text{令 } \mathbf{P}_K^{-1} = \Phi_K^T \cdot \Phi_K$$

$$\begin{aligned} \Phi_K^T \cdot \Phi_K &= \mathbf{P}_{K-1}^{-1} + \mathbf{x}_K^T \cdot \mathbf{x}_K \\ \mathbf{P}_K &= (\Phi_K^T \cdot \Phi_K)^{-1} = (\mathbf{P}_{K-1}^{-1} + \mathbf{x}_K^T \cdot \mathbf{x}_K)^{-1} \end{aligned}$$

之后来解决第二项

$$\begin{aligned}\Phi_K^T \cdot y_K &= \begin{pmatrix} x_1^T & x_2^T & \dots & x_K^T \end{pmatrix} \cdot \begin{pmatrix} y_1 \\ y_2 \\ \dots \\ y_K \end{pmatrix} \\ &= \sum_{i=1}^{K-1} x_i^T \cdot y_i + x_K^T \cdot y_K \\ &= \Phi_{K-1}^T \cdot y_{K-1} + x_K^T \cdot y_K\end{aligned}$$

对 ω_K 的表达式有

$$\begin{aligned}\omega_{K-1} &= (\Phi_{K-1}^T \cdot \Phi_{K-1})^{-1} \cdot (\Phi_{K-1}^T \cdot y_{K-1}) = P_{K-1} \cdot \Phi_{K-1}^T \cdot y_{K-1} \\ P_{K-1}^{-1} \cdot \omega_{K-1} &= \Phi_{K-1}^T \cdot y_{K-1}\end{aligned}$$

上述得到的结果重新代回 ω_K 的表达式

$$\begin{aligned}\omega_K &= (\Phi_K^T \cdot \Phi_K)^{-1} \cdot (\Phi_K^T \cdot y_K) \\ &= P_K \cdot (\Phi_{K-1}^T \cdot y_{K-1} + x_K^T \cdot y_K) \\ &= P_K \cdot (P_{K-1}^{-1} \cdot \omega_{K-1} + x_K^T \cdot y_K) \\ &= P_K \cdot ((P_K^{-1} - x_K^T \cdot x_K) \cdot \omega_{K-1} + x_K^T \cdot y_K) \\ &= \omega_{K-1} - P_K \cdot x_K^T \cdot x_K \cdot \omega_{K-1} + P_K \cdot x_K^T \cdot y_K \\ &= \omega_{K-1} - P_K \cdot x_K^T \cdot (x_K \cdot \omega_{K-1} - y_K)\end{aligned}$$

其中

$$P_K \cdot x_K^T = (P_{K-1}^{-1} + x_K^T \cdot x_K)^{-1} \cdot x_K^T$$

由Woodbury 恒等式

$$(A + BCD)^{-1} = A^{-1} - A^{-1}B(C^{-1} + DA^{-1}B)^{-1}DA^{-1}$$

如果令

$$\begin{aligned}A &= P_{K-1}^{-1} \\ B &= x_K^T \\ C &= I \\ D &= x_K\end{aligned}$$

可得出

$$P_K = P_{K-1} - P_{K-1} \cdot x_K^T \cdot (I + x_K \cdot P_{K-1} \cdot x_K^T)^{-1} \cdot x_K \cdot P_{K-1}$$

综上所述

$$\begin{aligned}\omega_K &= \omega_{K-1} - P_K \cdot x_K^T \cdot (x_K \cdot \omega_{K-1} - y_K) \\ P_K &= P_{K-1} - P_{K-1} \cdot x_K^T \cdot (I + x_K \cdot P_{K-1} \cdot x_K^T)^{-1} \cdot x_K \cdot P_{K-1} \\ P_K^{-1} &= \Phi_K^T \cdot \Phi_K\end{aligned}$$

因此得到了最小二乘法的参数更新公式

Question 2 用递归最小二乘法理解DFP方法

牛顿法是一种用二阶导数加速收敛的迭代优化算法。

对一维函数来说，用切线不断逼近函数的根。

$$x_{k+1} = x_k - \frac{f(x_k)}{f'(x_k)}$$

拓展到n维后牛顿法的迭代公式为：

$$x_{k+1} = x_k - \nabla^2 f(x_k)^{-1} \nabla f(x_k)$$

但是实际运算时Hessian矩阵运算量大，为解决此问题推导出了拟牛顿法，即把Hessian矩阵也写作迭代的形式，用当前迭代点的梯度信息来获得Hessian矩阵的近似值。

假设在k迭代点有：

$$\begin{aligned} x_{k+1} &= x_k - H_k \nabla f(x_k) \\ H_k &= \nabla^2 f(x_k)^{-1} \end{aligned}$$

H_k 有如下迭代形式 $H_{k+1} = H_k + \epsilon$

关键在于 ϵ 的求解。

对Hessian矩阵，他是一个对称矩阵 $H^T = H$ ，有 $(H^{-1})^T = (H^T)^{-1} = H^{-1}$ ，他的逆矩阵也是对称矩阵， H_k 是对称的，于是这个增量 ϵ 可用一个对称矩阵表示。

对DFP方法，令 $\epsilon = \beta uu^T + \gamma vv^T$

$$H_{k+1} = H_k + \beta uu^T + \gamma vv^T$$

两边乘上 y_k

$$\begin{aligned} s_k &= H_{k+1} y_k = H_k y_k + \beta uu^T y_k + \gamma vv^T y_k \\ s_k - H_k y_k &= \beta uu^T y_k + \gamma vv^T y_k \end{aligned}$$

令 $u = s_k$ ， $v = H_k y_k$ ， $\beta uu^T y_k = s_k$ ， $\gamma vv^T y_k = -H_k y_k$ ，用乘法结合律先计算 $s_k^T y_k$ 为一个数，然后再与 β 组合，后面求 γ 同理

$$\begin{aligned} \beta s_k s_k^T y_k &= s_k \\ \beta s_k^T y_k &= 1 \\ \beta &= \frac{1}{s_k^T y_k} \\ \gamma (H_k y_k) (H_k y_k)^T y_k &= -H_k y_k \\ \gamma (H_k y_k)^T y_k &= 1 \\ \gamma &= \frac{1}{(H_k y_k)^T y_k} = \frac{1}{y_k^T H_k y_k} \end{aligned}$$

回代后得到

$$\begin{aligned} H_{k+1} &= H_k + \frac{s_k s_k^T}{s_k^T y_k} - \frac{H_k y_k y_k^T H_k}{y_k^T H_k y_k} \\ s_k &= H_{k+1} y_k \end{aligned}$$

先消去 H_{k+1} 反解 s_k 再正向计算得到 H_{k+1}

和递推法最小二乘法的推导思路类似，都是先把想要变为迭代值表示的值写作 $X_{k+1} = X_k + \epsilon$ 的形式，然后通过分析 X_{k+1} 自身展开的表达式来得到用先前的量表示的、相较于矩阵求逆运算量小的增量 ϵ 的表示形式

Question 3 阅读有关支持向量机和KKT条件相关的资料

KKT条件

KKT条件用于约束优化问题：

对等式约束优化问题

$$\begin{aligned} \min \quad & f(x) \\ \text{s.t.} \quad & g(x) = 0 \end{aligned}$$

令 $L(x, \lambda) = f(x) + \lambda g(x)$, 转为无约束问题

$$\min_{x, \lambda} L(x, \lambda)$$

求偏导数便可以得到最优值

$$\begin{aligned} \nabla_x L(x, \lambda) &= \nabla f(x) + \lambda \nabla g(x) = 0 \\ \nabla_\lambda L(x, \lambda) &= g(x) = 0 \end{aligned}$$

对不等式约束优化问题

$$\begin{aligned} \min \quad & f(x) \\ \text{s.t.} \quad & g(x) \leq 0 \end{aligned}$$

称 $g(x) \leq 0$ 为原始可行性，定义约束条件构成的集合为可行域 K ，假设 x^* 为最优解。

如果 $g(x^*) < 0$ ，最优解落在可行域内，称为内部解，此时约束条件无效，最优解满足 $\nabla f(x^*) = 0, \lambda = 0$

如果 $g(x^*) = 0$ ，最优解落在可行域边界，称为边界解，此时约束不等式变为等式， $\nabla f = -\lambda \nabla g$ ，此时 ∇f 指向 K 内部，因为最小值在边界取到，可行域内部值应大于等于边界值，而 ∇g 指向 K 外部，因为约束是小于等于0的，因此 $\lambda \geq 0$ ，此称为对偶可行性。

不论是内部解还是外部解，都有 $\lambda g(x) = 0$ ，称为互补松弛性。

最佳解的必要条件包括Lagrangian函数的定常方程式、原始可行性、对偶可行性、互补松弛性

对偶可行性取决于优化目标，如果是最大化 $f(x)$ 而 $g(x) \leq 0, \lambda \leq 0$

因此对如下优化问题

$$\begin{aligned} \min \quad & f(x) \\ \text{s.t.} \quad & g_j(x) = 0, j = 1, \dots, m \\ & h_k(x) \leq 0, k = 1, \dots, p \end{aligned}$$

构建Lagrangian函数

$$L(x, \lambda_j, \mu_k) = f(x) + \sum_{j=1}^m \lambda_j g_j(x) + \sum_{k=1}^p \mu_k h_k(x)$$

KKT条件包括

$$\begin{aligned} \nabla_x L &= 0 \\ g_j(x) &= 0, j = 1, \dots, m \\ h_k(x) &\leq 0, \\ \mu_k &\geq 0, \\ \mu_k h_k(x) &= 0, k = 1, \dots, p \end{aligned}$$

支持向量机

二分类问题，对于线性可分的数据，支持向量机目的是找一个最优超平面将数据分为两组，最优体现在此超平面与两侧离他最近的数据点(支持向量)保持着最大的间隔，如果在数据的特征维度内找不到这样的线性的超平面，那么先升维知道存在一个超平面可以线性二分，然后再将此超平面投影到原本数据的特征维度上。

Question 4 了解投影算子，并构造一个投影算子，将任意向量 v 投影到由 $Ax=b$ 确定的平面上

投影算子是向量空间 V 上的一个线性算子，满足 $P^2 = P$ ，如果一个投影算子的投影方向是正交于被投影的平面的，则称之为正交投影算子。 $P^T = P$ ，是个对称矩阵。

投影算子满足

$$\begin{aligned} AP(v) &= b \\ v - P(v) &\perp \{x | Ax = 0\} \end{aligned}$$

构造 $P(v) = v - A^T(AA^T)^{-1}(Av - b)$

$$\begin{aligned} AP(v) &= Av - AA^T(AA^T)^{-1}(Av - b) = b \\ v - P(v) &= A^T(AA^T)^{-1}(Av - b) \end{aligned}$$

这个式子后面两项均为常数，视为 C

$$(v - P(v))^T \cdot x = C \cdot Ax = 0$$

满足正交性。

所以投影算子为：

$$\begin{aligned} P(v) &= Pv + c \\ P &= I - A^T(AA^T)^{-1}A, \quad c = A^T(AA^T)^{-1}b \end{aligned}$$